

Informe-Meeple

Diseño e Implementación de un Lenguaje

Informe final del Proyecto Especial · Noviembre de 2026

Nombres	Apellidos	Legajos	E-mails
Carolina	Goldbaum	65112	cgoldbaum@itba.edu.ar
Maria	Resek	65528	mresek@itba.edu.ar
Santiago	Fraga	64346	safraga@itba.edu.ar
Ivonne	Choe	64880	ichoe@itba.edu.ar

Equipo G-145 · Repositorio: https://github.com/cgoldbaum/meeple-compiler (rama `development`)

Tabla de contenidos:

1. Introducción 2
2. Modelo Computacional 2
   - 2.1. Dominio 2
   - 2.2. Lenguaje 2
3. Implementación 4
   - 3.1. Frontend 4
   - 3.2. Backend 4
   - 3.3. Adicionales 6
   - 3.4. Dificultades Encontradas 6
4. Futuras Extensiones 7
5. Conclusiones 7
6. Referencias 7
7. Bibliografía 7

1. Introducción

Meeple es un lenguaje para escribir las reglas de un juego de mesa por turnos y la forma en que deciden sus jugadores. Un programa declara el juego, una o más estrategias y una tanda de partidas; el resultado es un reporte con estadísticas sobre esas partidas: el porcentaje de victorias de cada estrategia, cuánto dura una partida (promedio, mínimo, máximo) y qué valores toman las métricas, que son definidas por quien programa. De esta manera, se puede ver si un juego está balanceado, o si una forma de jugar domina a otra, sin jugar mil partidas en la mesa.

El compilador, escrito en C con Flex y Bison sobre el proyecto base de la cátedra [2], valida el programa y emite un programa en C que, compilado junto con un *runtime* propio, corre las simulaciones. Este informe cubre el lenguaje tal como quedó después de la devolución del Stage I [1] y las decisiones de implementación. Las instrucciones de uso están en el README del repositorio.

2. Modelo Computacional

2.1. Dominio

Juegos por turnos con estos componentes: jugadores, un tablero lineal de celdas numeradas, fichas, dados con caras arbitrarias y mazos de cartas cuyo formato define el programa. Un juego se describe con su preparación, lo que pasa en cada turno, la condición de victoria y un límite de turnos.

Los jugadores son agentes simulados. En el turno, el juego les hace preguntas tipadas, llamadas *decisiones* (“qué carta jugar”, “qué ficha mover”). Una *estrategia* asigna a cada decisión una *política*: una regla que elige entre las opciones ordenándolas por uno o más *criterios*. Comparar estrategias es comparar políticas sobre las mismas reglas.

Los sustantivos del dominio son tipos, los verbos son acciones y las relaciones se escriben con *conectores de composición* (`in`, `on`, `to`, `from`, `of`, `per`, `by`, `where`, `uses`): un mazo es *of* un tipo de carta, una mano existe *per player*, un jugador *uses* una estrategia. Quedan afuera los tableros con forma de grafo, la información oculta, los equipos y los empates entre ganadores.

2.2. Lenguaje

Un programa es una secuencia de bloques `game`, sentencias `simulate` y bloques `report`. Cada `report` resume las partidas del `simulate` inmediatamente anterior.

El Código 1 es un programa completo: cada jugador tiene dos fichas y en cada turno decide cuál mover. `Rezagada` mueve la más atrasada y `Puntera`, la más adelantada.

```
game "Dos fichas" {
    players 2 to 2;
    board cells 20;
    die D6 faces 1 to 6;
    piece a per player;
    piece b per player;
    decision mover: piece;
    strategy Rezagada { mover: prefer min option.cell; }
    strategy Puntera  { mover: prefer max option.cell, random; }
    prepare { players[0] uses Rezagada; players[1] uses Puntera; }
    turn {
        piece f = ask current for mover(current.pieces);
        move f forward roll D6;
    }
    win when current.a.cell == 19 and current.b.cell == 19;
    end after 200 turns;
}
simulate 1000 games of "Dos fichas" with 2 players seed 7;
report { winrate by strategy; avg turns; }
```

Código 1: Un programa Meeple completo.

La línea `decision mover: piece;` declara la pregunta y su tipo. Cada `strategy` le da una política, y

`ask current for mover(...)` la hace con las fichas del jugador como opciones. Dentro de una política, `option` es la opción que se está evaluando, y `random` desempata.

El reporte (Código 2) muestra algo que no es obvio al leer las reglas: `Puntera` nunca gana. Siempre mueve la ficha más adelantada: cuando esta llega a la última celda la sigue eligiendo, así que la otra no avanza nunca.

```
simulate 1000 games of "Dos fichas" with 2 players seed 7
winrate by strategy
  Rezagada: 100.00%
  Puntera: 0.00%
avg turns: 22.59
```

Código 2: Salida del programa del Código 1.

Dentro de `game` se declaran los componentes, las variables del juego (que viven toda la partida y que las estrategias leen), las métricas (`metric nombre = expr;`), las decisiones y estrategias, y los bloques `prepare` y `turn`. Las sentencias son declaraciones tipadas, asignación, `if`, `while`, `repeat` y `for`, más ocho acciones del dominio (`place`, `move`, `shuffle`, `draw`, `play`, `give`, `take` y `log`). Además de la aritmética y la lógica, las expresiones tienen las agregaciones `count`, `sum`, `max`, `min` y `select`, con filtro `where` opcional.

Cambios por la devolución. Toda la sintaxis nueva entró en el Stage II, para que el *backend* trabajara sobre la gramática final (Tabla 1). Los números de la columna "Punto" de las Tablas 1 y 2 son los de la devolución del Stage I.

Punto	En la especificación	En el lenguaje final
3, 4	Cada carta del mazo, con todos sus campos	Producto cartesiano de generadores (`valor: 1 to 12; palo: {...};`) y cartas posicionales (`card {1, 1}`)
5	`for` solo sobre colecciones	También sobre un rango: `for (i in 1 to n)`
8	Un juego sin `win when` se rechazaba	Es válido: termina por el límite de turnos
9, 30	`seed` y `log level` sueltos	Cláusulas de `simulate`: `... seed 42 verbose;`
10	`decision jugar(opciones: Carta[]) -> Carta;`	`decision jugar: Carta;`
13	Las estrategias no veían el estado del turno	Variables del juego, de solo lectura para las estrategias
14	`players 2 to 4;`	También un conjunto: `players {2, 4, 6};`
16, 31	Un único criterio por política	Criterios encadenados que desempatan, filtro `where` y criterio `input` (decide una persona)
23, 29	`setup`, `turn_number`	`prepare`, `turns`
25	`log "{1} {2}", a, b;`	Interpolación: `log "{current.name} avanzo {pasos}";`
7, 27	`winrate` y `avg turns` fijos	`metric` propias, con `avg`, `min` y `max` en el `report`
28	`board Tablero cells 63;`	`board cells 63;`: hay un solo tablero, y el nombre sobraba

Tabla 1: Cambios introducidos a partir de la devolución del Stage I.

`decision` y `strategy` siguen separados (punto 33). `decision` es el contrato común: fija una vez el tipo de `option` y permite rechazar al compilar una estrategia incompleta o un `ask` a una decisión inexistente. Las decisiones existen aunque no haya estrategias, porque un jugador sin `uses` responde con `prefer first`. Con la firma nueva, cada una ocupa una línea.

Semántica. La especificación dejaba abiertos varios puntos de ejecución. Los fijamos antes de implementar:

- `turns` vale 0 en `prepare` y 1 en el primer turno. Sin `end after`, el límite es de 1000 turnos, así que no hay simulaciones infinitas. Sin `seed`, la semilla es 0.
- `clockwise` recorre los jugadores 0, 1, …, N−1; `counterclockwise` empieza también por el 0 y sigue por N−1, N−2, …

- Jugadores, fichas, cartas y mazos son referencias: asignarlos copia la referencia y == compara identidad. Los arreglos, en cambio, se copian al asignarlos.
- Si el `where` de una política descarta todas las opciones, `ask` devuelve `none` cuando la decisión es sobre una referencia, y es un error de ejecución cuando es sobre `integer`, `string` o `boolean`, que no tienen `none`.
- Un `log` evalúa siempre sus interpolaciones y solo imprime con `verbose`, así que la traza no cambia el reporte aunque una interpolación tire un dado.
- `winrate by strategy` divide las victorias por la cantidad de jugadores que usaron la estrategia, no por partidas. Se cuenta la estrategia que cada jugador tenía al terminar la partida. Si los dos jugadores usan la misma y todas las partidas tienen ganador, da 50 %. `winrate`, sin `by`, es el porcentaje de partidas con ganador.

La Tabla 2 contesta las preguntas de la devolución que no resuelve la Tabla 1.

Punto	Pregunta	Respuesta
2	¿Distingue mayúsculas?	Sí.
5	¿`repeat n` expone un índice?	No. Ejecuta el bloque `n` veces; para un índice está el `for` sobre un rango.
6	¿`clockwise` es ascendente?	Sí (ver arriba).
11	¿Una estrategia puede devolver algo fuera de las opciones?	No: una política solo elige entre las opciones del `ask`.
12	¿Qué hace `select`?	Devuelve, en el mismo orden, los elementos que cumplen el `where`.
15	¿Puede haber varios ganadores?	No en esta versión (Sección 4).
24	¿Cómo se propaga `none`?	No se propaga: usar un miembro de `none` es un error de ejecución con su línea. `max` y `min` de una colección vacía dan `none` si los elementos son referencias, y un error de ejecución si son valores.
32	¿Hay decisiones sobre fichas o jugadores?	Sí: el tipo de una decisión puede ser `piece` o `player` (Código 1).

Tabla 2: Respuestas a las preguntas de la devolución del Stage I.

3. Implementación

3.1. Frontend

Léxico. Lo único no rutinario es la interpolación del `log`. `"a {x} b"` se parte en un *token* de texto hasta la llave, la expresión `x`, que se analiza como cualquier otra, y otro *token* de texto hasta el cierre. Para eso el analizador léxico entra en un contexto inclusivo `INTERPOLATION` al abrir la llave y sale al cerrarla. Fuera de un `log`, una cadena con llaves es un error de sintaxis.

Sintáctico. La gramática tiene 105 terminales del lenguaje, 67 no terminales y 221 producciones, y ningún conflicto: el *build* corre Bison con `-Werror=conflicts-sr` y `-Werror=conflicts-rr`. Solo las expresiones son ambiguas a propósito, y se resuelven con precedencias. Las llaves son obligatorias en todo cuerpo, lo que elimina el *dangling else*, y no hay sentencias-expresión sueltas: `roll D6;` no es una sentencia.

AST. Tiene 21 estructuras. Cada sentencia, ítem del juego y expresión guarda su línea, que después usan los mensajes semánticos y los errores de ejecución. Las cadenas que duplica el analizador léxico pasan a ser del *parser*, y cada acción de Bison las guarda en el árbol o las libera; los `%destructor` cubren solo el camino de error.

3.2. Backend

3.2.1. Análisis semántico

Corre en tres pasadas. La primera junta los nombres de los `game`, porque un `simulate` puede aparecer antes del juego que nombra. La segunda recorre cada `game` en orden: como hay que declarar

antes de usar, alcanza con una pasada por juego, y al cerrarlo se chequean las reglas que miran hacia adelante, como que toda estrategia resuelva todas las decisiones. La tercera valida `simulate` y `report`.

La tabla de símbolos es una pila de alcances: programa, juego, bloque y agregación. En vez de producir una estructura intermedia, el análisis anota el AST: cada expresión recibe su tipo y qué es cada nombre (variable local o del juego, dado, mazo, ficha, estrategia, campo de carta). Así la tabla se libera al terminar y el generador solo lee el árbol. Una expresión mal tipada recibe un tipo `error` compatible con todo, para que un error no arrastre otros.

```
[ERROR][SemanticAnalyzer] Linea 2: "Juan" no esta declarado.
```

Código 3: Mensaje para `give 1 to Juan;` sin declarar `Juan`.

Además de los casos de la especificación, se rechazan más de cincuenta situaciones, cada una con su test: nombres repetidos o que tapan a otro, cartas incompletas, `option` fuera de una política, `current` en `prepare` (todavía no hay jugador activo) y `ask` dentro de una política, porque una estrategia que consulta a otra podría no terminar. Los casos 7 y 11 de la especificación ya no existen: el `log` posicional desapareció, y un juego sin `win when` es válido.

3.2.2. Generación de código

Decidimos generar C, aunque un intérprete sobre el AST habría sido más simple, porque una pregunta de balance necesita muchas partidas: el programa generado corre 300.000 partidas de La Oca con memoria constante. Además, el resultado es un ejecutable que se puede volver a correr sin el compilador.

El generador escribe el C en el archivo que recibe por argumento. Cada `game` se traduce a declaraciones `static` y funciones; `main` contiene los `simulate` y los `report` en el orden del programa. Lo que declara el juego *k* lleva el prefijo `g<k>_`, lo que agrega el compilador `h<k>_`, las locales `v_` y los temporales `_t<n>`; así, una variable Meeple llamada `main` o `int` es válida (Tabla 3).

Meeple	C generado
`cardtype Carta { ... }`	Un `struct` con un campo por cada campo de la carta
`deck`, `piece ... per player`	Una lista, o un arreglo indexado por jugador, que se rearma al empezar cada partida
`prepare`, `turn`, `win when`	Una función cada uno; todas las `metric` comparten una
`for (x in C)`	Un bucle sobre una copia de `C`, para que el cuerpo pueda modificar `C`
`count`, `sum`, `max`, `min`, `select`	Una *statement expression* de GCC, `({ ... })`, con temporales propios
`a.campo` sobre una referencia	`mp_check(a, línea)->campo`, que falla con la línea si `a` es `none`
`/`, `%`, `board.cell(i)`, `a[i]`	Llamadas al *runtime* que validan el operando y reciben la línea
`strategy`, `decision`	Una tabla de políticas por decisión, indexada por la estrategia del jugador

Tabla 3: Traducción de las construcciones principales.

Las agregaciones son expresiones que necesitan un bucle. Con las *statement expressions* quedan en el lugar donde aparecen, y una agregación anidada genera bucles anidados con temporales distintos. A cambio, el C generado requiere GCC o Clang.

Las estrategias se traducen a datos (Código 4). Cada criterio es una función que puntúa una opción, y cada decisión tiene una tabla con una política por estrategia; la posición 0 es "sin estrategia", equivalente a `prefer first`, y `mp_ask` indexa la tabla con la estrategia actual del jugador.

```
static int h0_key0_0(MpValue option, MpPlayer * current) {
    return ((MpPiece *) mp_check(((MpPiece *) (option).p), 8))->cell;
}
static const MpRanking h0_rankings0[] = { { h0_key0_0, false } };
static const MpPolicy h0_policies_mover[] = {
    { NULL, NULL, 0, MP_FIRST },
    { NULL, h0_rankings0, 1, MP_FIRST },
    { NULL, h0_rankings1, 1, MP_RANDOM }
};
static const MpDecision g0_mover = { "mover", true, mp_describe_piece, h0_policies_mover };
```

Código 4: C generado para la decisión del Código 1 (se omiten el criterio de `Puntera` y los `(void)`).

El C generado compila con `-Wall -Wextra -Werror`. Para eso se emite `(void) v_x;` después de cada local y se desactivan `-Wunused-function` y `-Wunused-const-variable`, porque un juego puede declarar componentes que no usa.

3.2.3. Runtime

`MeepleRuntime.c` implementa el ciclo de una partida, las listas, las acciones, `ask`, las estadísticas y el reporte. Está en C99 con extensiones de GNU (`__typeof__`, `__alignof__`) y no usa ninguna biblioteca externa. `simulate.sh` lo copia junto al C generado y compila los dos con `gcc -std=gnu99`, así que se puede corregir sin reconstruir el compilador.

- **Azar.** Un generador propio, con muestreo por rechazo para que elegir entre n opciones no tenga sesgo. No usamos `rand()` porque su secuencia depende de la biblioteca de C. Con el mismo compilador de C, la misma semilla da la misma simulación en Docker, en el CI o en cualquier otra máquina.
- **Memoria.** Las cartas, las listas y los resultados de `select` salen de una *arena* que se vacía al empezar cada partida. El código generado nunca libera memoria y no puede perderla, y las estadísticas son acumuladores, así que la memoria no crece con la cantidad de partidas.
- **Errores.** Un acceso a `none`, un índice fuera de rango, una división por cero o una celda inexistente imprimen `Error de ejecucion en la linea N: ...` y terminan con código 1, sin reporte: seguir falsearía las métricas.

3.3. Adicionales

- **`prefer input`:** una decisión la toma una persona. El programa muestra la partida, el turno y las opciones numeradas, y vuelve a preguntar si la respuesta no es válida.
- **Pruebas de punta a punta:** `e2e.sh` genera el C de los 49 programas aceptados y de 8 que fallan al ejecutarse, y lo compila con `-Werror`, AddressSanitizer y UndefinedBehaviorSanitizer. En 19 casos compara la salida con una esperada; varias se calcularon a mano con programas sin azar. Lo corre el CI junto con los 135 *tests* de aceptación y rechazo.

3.4. Dificultades Encontradas

Un tipo y una asignación que empiezan igual. En un turno, `Carta[] jugables = ...;` declara y `Mazo[0].valor = 9;` asigna. La gramática natural tenía un conflicto sobre `[` que, resuelto por desplazamiento, rompía las asignaciones. Factorizamos a izquierda las sentencias que empiezan con un identificador para que decida el *token* que sigue al `[`. Descartamos que el analizador léxico distinguiera los nombres de tipo de los demás identificadores, porque eso lo acoplaría a la tabla de símbolos.

Lo que no se puede chequear al compilar. `players[k]` puede ser válido con cuatro jugadores y no con dos, y una mano puede estar vacía recién a mitad de partida. Para cada caso de este tipo decidimos si es un error de ejecución con línea o un comportamiento definido. Por ejemplo, `draw` sobre un mazo con menos cartas de las pedidas mueve las que hay, y `move` se detiene en la última celda.

Un ejemplo de la especificación sin ganador. Al fijar la salida esperada del ejemplo 6.2 vimos que nadie gana nunca: con cuatro cartas repartidas de a tres, el puntaje no pasa de 8 y la condición pide 15. El error estaba en nuestra especificación, no en el compilador, y lo encontramos al implementarlo.

El orden de evaluación de C. C no define el orden de los argumentos de una función, así que las

cartas se arman campo por campo: de lo contrario, `card { a: roll D; b: roll D; }` podría dar resultados distintos. Los operandos de una expresión (`roll D - roll D`) quedan en el orden que elige el compilador de C, y por eso la reproducibilidad vale con el mismo compilador.

4. Futuras Extensiones

- Equipos y varios ganadores (punto 15).
- Agregar y quitar elementos de un arreglo; hoy solo se arman con `select` o copiando un mazo.
- Un orden de evaluación fijo para los operandos, para que la reproducibilidad no dependa del compilador de C.
- La columna en los mensajes de error.
- Tableros con forma de grafo y casillas con efectos, que hoy se programan con `if` sobre la celda.

5. Conclusiones

Lo que más ordenó el trabajo fue fijar la sintaxis completa en el Stage II y escribir la semántica antes de implementarla: casi todas las dudas de la última etapa eran del tipo "qué pasa si…", y tenerlas respondidas evitó rehacer código. El generador lo probamos ejecutando lo que genera. Compilar cada programa de prueba con *sanitizers* detecta errores de memoria y comportamiento indefinido en el C generado, y comparar su salida mostró, por ejemplo, que el 6.2 no tenía ganador.

La limitación principal es que Meeple compara estrategias sobre reglas fijas, con un solo ganador por partida y sin información oculta. Eso alcanza para juegos de carrera y de cartas simples, pero no para juegos de equipos o de *bluff*.

6. Referencias

1. Equipo G-145. *Especificación de Meeple* (Stage I), 2026. `doc/Especificacion-Meeple.pdf`.
2. A. Golmar. *Flex-Bison-Compiler*, proyecto base de la cátedra. https://github.com/Alpha-Theta-Gamma-Mu/Flex-Bison-Compiler

7. Bibliografía

1. Cátedra de Autómatas, Teoría de Lenguajes y Compiladores (ITBA). *Reglas y Stages del Proyecto Especial* (2026), *Análisis Semántico* (v0.2.0), *Generación de Código* (v0.1.0) y *Runtime* (v0.1.0).
2. V. Paxson, W. Estes, J. Millaway. *Lexical Analysis With Flex*, manual de la versión 2.6.4.
3. C. Donnelly, R. Stallman. *Bison: The Yacc-compatible Parser Generator*, manual de la versión 3.8.